

◆ Step 1: Automated Infrastructure with Terraform

We wrote an Infrastructure-as-Code (IaC) blueprint to provision your cloud foundation safely within the **AWS Free Tier** limits using t3.micro instances:

- **VPC & Networking:** Created a custom Virtual Private Cloud with isolated subnets and route tables so your nodes can communicate securely.
- **IAM Security Framework:** Set up least-privilege Identity & Access Management (IAM) roles allowing the EKS control plane to manage network interfaces and worker nodes to securely pull images.
- **ECR Registry:** Set up an Amazon Container Registry (demo-app) to house your private Docker container images.

◆ Step 2: Bypassed Corporate Proxy Controls (Offline Mirroring)

Since your corporate network restricts direct access to public image registries like registry.terraform.io, we built an entirely local, isolated execution layout:

- We created a centralized local directory on your machine (C:\tools\plugins\).
- We mapped out an exact nested file-tree layout down to the platform architecture target (...\\windows_386\).
- We wrote a custom **config.tfrc** mapping file and injected the `$env:TF_CLI_CONFIG_FILE` variable, forcing Terraform to instantly locate your local 5.0.0 provider .exe binary without touching the internet.

◆ Step 3: Git Workspace Optimization

We cleaned your source control tracking so you don't accidentally push massive binary chunks or corporate network configs to your code repository:

- Configured a custom **.gitignore** profile to exclude heavy executable caches (.terraform/) while allowing you to push documentation and diagrams (*.pdf).
- Optimized your local Git instance buffer constraints (`http.postBuffer = 500MB`) to maintain stable tracking data transfers.

◆ Step 4: Deployed Live Application Workloads

We built a multi-tier Kubernetes manifest (sample-app.yaml) to run a live web application:

- **Namespace:** Created an isolated workspace partition (sample-app) to keep things organized.

- **Deployment:** Created a highly available controller managing 2 application pod replicas running your containerized HTML application.
- **LoadBalancer Service:** Wired a public-facing service that commands AWS to provision a hardware load balancer, giving you a live, external URL to access your app from a web browser.

2. Advanced Things You Can Check on Your Cluster Right Now

Now that your cluster is operational, you can use these kubectl commands to inspect exactly how Kubernetes manages your compute grid under the hood.

Check 1: Inspect Detailed Resource Utilization

To find out exactly how much CPU and memory your worker nodes or individual pods are consuming, use the top metrics commands:

PowerShell

Check current CPU and Memory load on your physical AWS instances

kubectl top nodes

Check resource consumption across your application pods

kubectl top pods -n sample-app

Check 2: Describe Cluster Resources (The Ultimate Debugger)

If a pod gets stuck, fails to boot, or your load balancer behaves oddly, you can pull the raw system logs and events directly from the API layer:

PowerShell

Read the complete lifecycle events, target status, and network layout of a pod

kubectl describe pod -l app=sample-web -n sample-app

Inspect the live configuration and active endpoints of your LoadBalancer

kubectl describe service sample-app-service -n sample-app

Check 3: Read Live Application Container Logs

If your application container is running but displaying errors, you can stream the standard output (stdout) console logs directly out of the container engine:

PowerShell

Get the names of your running pods

```
kubectl get pods -n sample-app
```

Stream logs live from a specific pod container (replace with your pod's exact name)

```
kubectl logs <pod-name> -n sample-app --tail=50
```

Check 4: Execute an Interactive Shell inside a Running Pod

Just like SSHing into a Linux server, you can securely execute a remote terminal terminal container shell inside Kubernetes to verify internal paths or test database connections natively:

PowerShell

Drop into an interactive bash/sh session inside your application container

```
kubectl exec -it <pod-name> -n sample-app -- /bin/sh
```

Inside the pod shell, verify the local index file is serving properly, then type 'exit'

```
cat /usr/share/nginx/html/index.html
```

```
exit
```

Check 5: Test High Availability (Simulate a Pod Crash)

Kubernetes is self-healing. Since we set replicas: 2 in your manifest, the deployment controller will instantly spin up a replacement if a pod dies. Let's test it:

PowerShell

1. Watch your pods in real-time in a separate window

```
kubectl get pods -n sample-app -w
```

2. Delete one of your running pods forcefully

```
kubectl delete pod <pod-name> -n sample-app
```